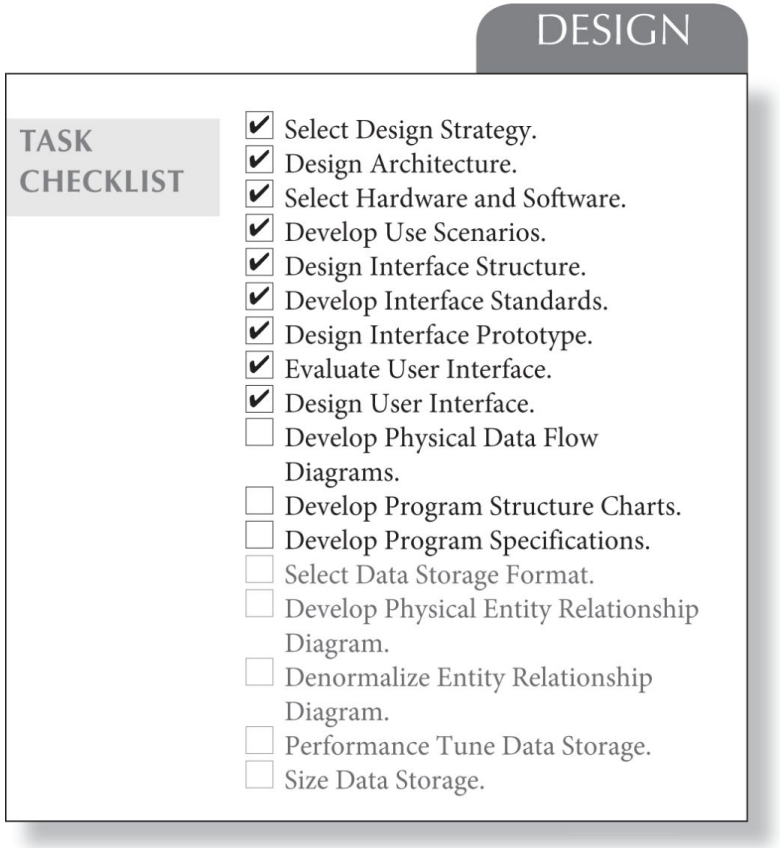
Program Design

How do programmers know **what to program** and **how to organize** their code?


DESIGN
TASK CHECKLIST
Select Design Strategy.
Design Architecture.
Select Hardware and Software.
Develop Use Scenarios.
Design Interface Structure.
Develop Interface Standards.
Design Interface Prototype.
Evaluate User Interface.
Design User Interface.
Develop Physical Data Flow Diagrams.
Develop Program Structure Charts.
Develop Program Specifications.
Select Data Storage Format.
Develop Physical Entity Relationship Diagram.
Denormalize Entity Relationship Diagram.
Performance Tune Data Storage.
Size Data Storage.

Logical vs. Physical DFDs

- Logical DFDs → **business view** of the system (no implementation details)
- Physical DFDs → **systems view (technical view)** of the system. Includes implementation details such as:
  - references to actual technology
  - The format of information → what do the data flows carry?
  - Human interaction involved → In which places, human interaction is required?

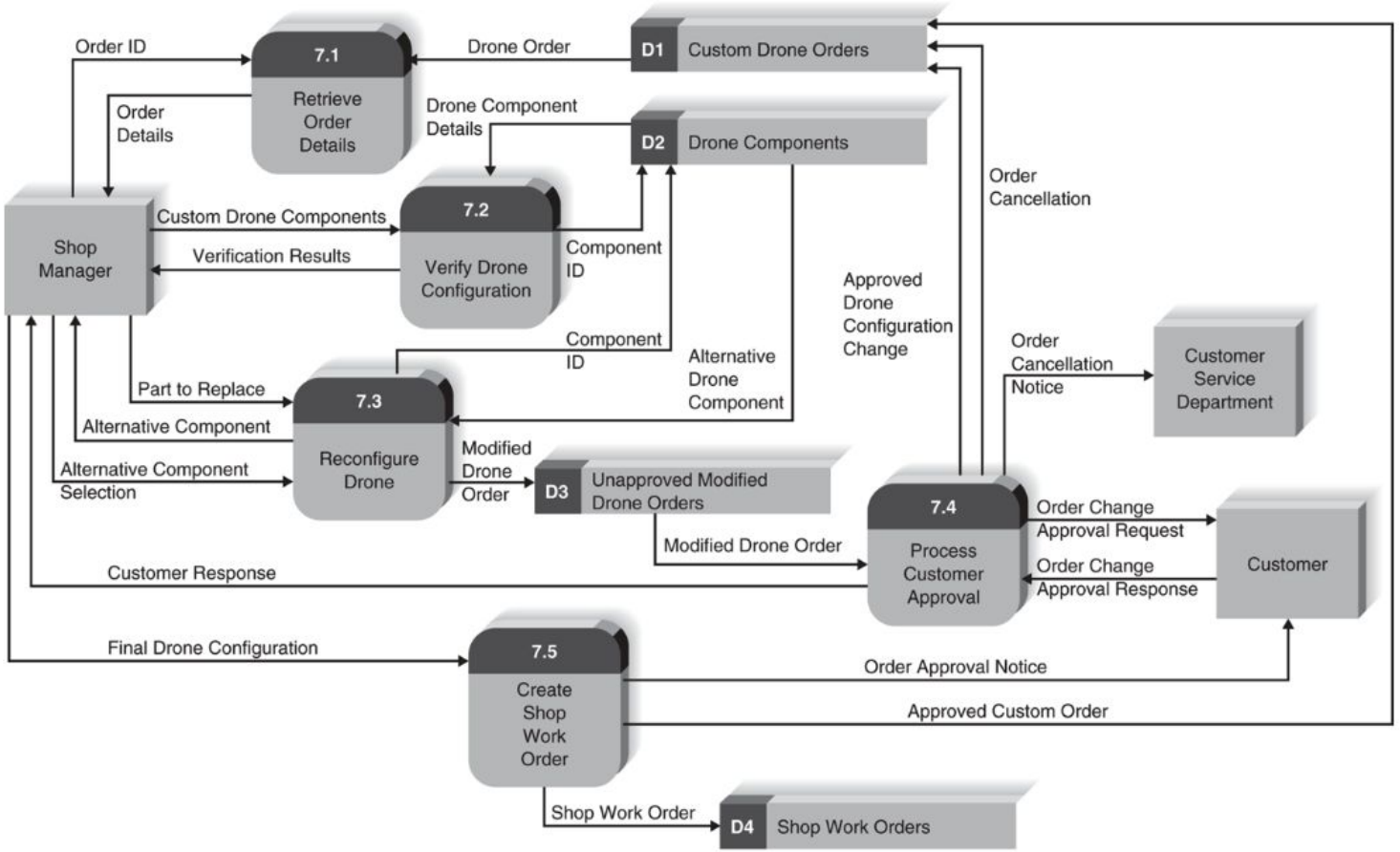


FIGURE 4-12 Shop manager approval of custom drone order level 1 DFD.

Creating Physical Data Models

1. Add implementation references

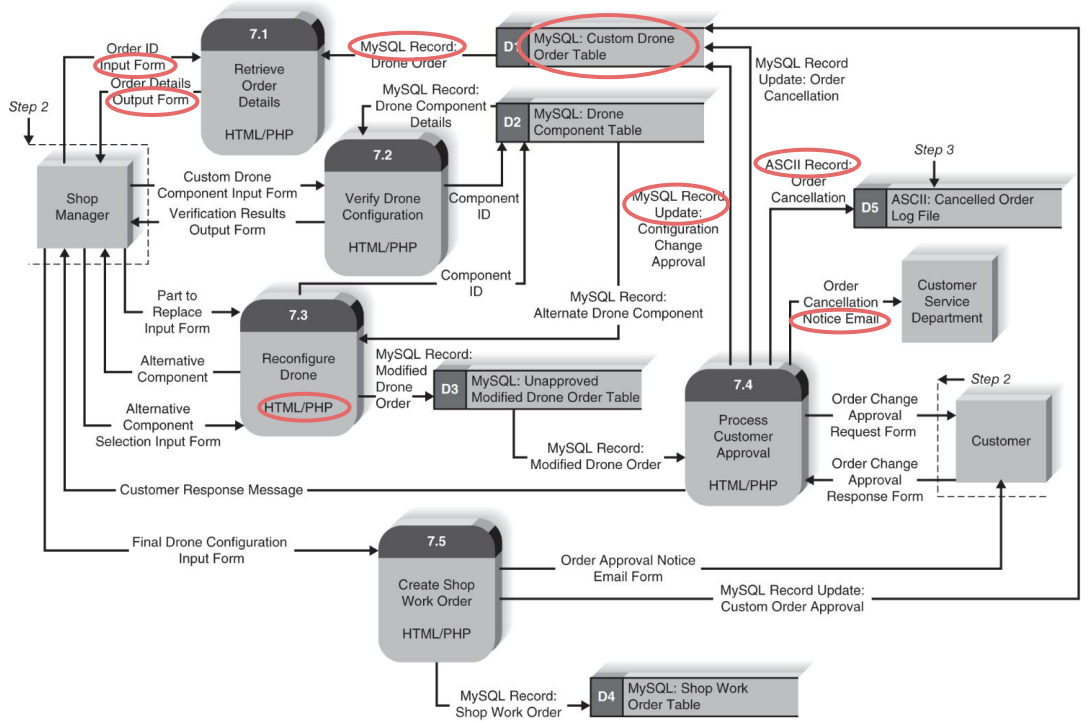


FIGURE 9-2 Physical DFD for shop manager approval of custom drone order level 1 DFD.

Creating Physical Data Models

1. Add implementation references
2. Draw a human-machine boundary
   a. Notice that not all external entities have a boundary around them. Why?

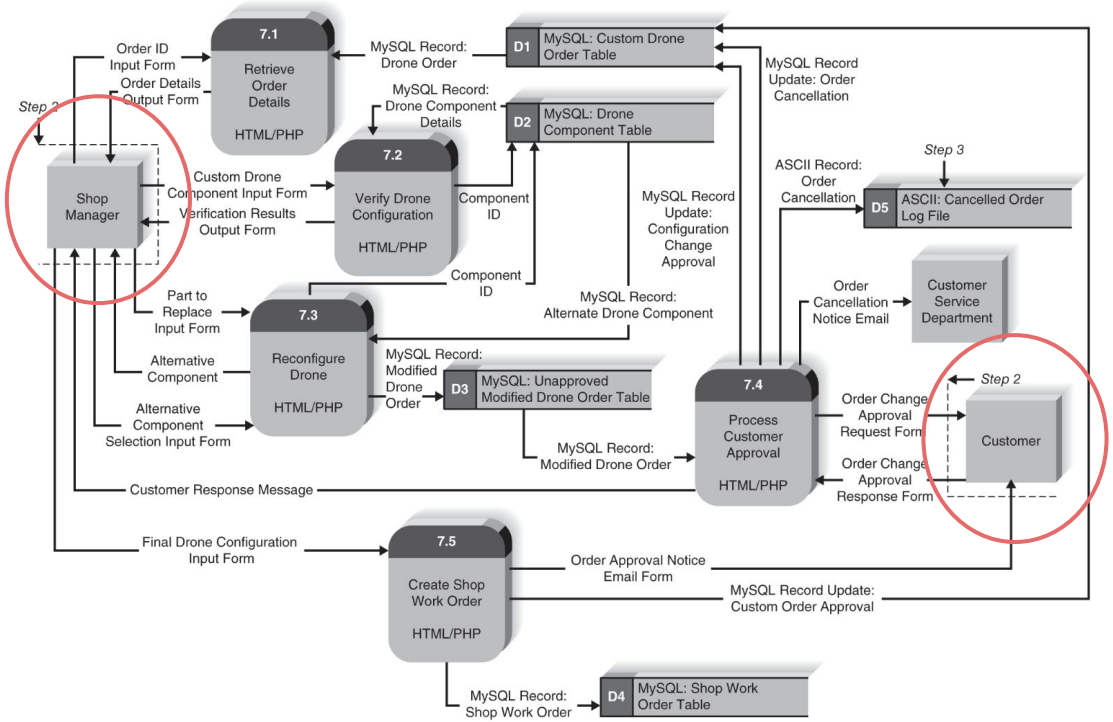


FIGURE 9-2 Physical DFD for shop manager approval of custom drone order level 1 DFD.

Creating Physical Data Models

1. Add implementation references
2. Draw a human-machine boundary
   a. Notice that not all external entities have a boundary around them. Why?
3. Add system related date stores, data flows, and processes.

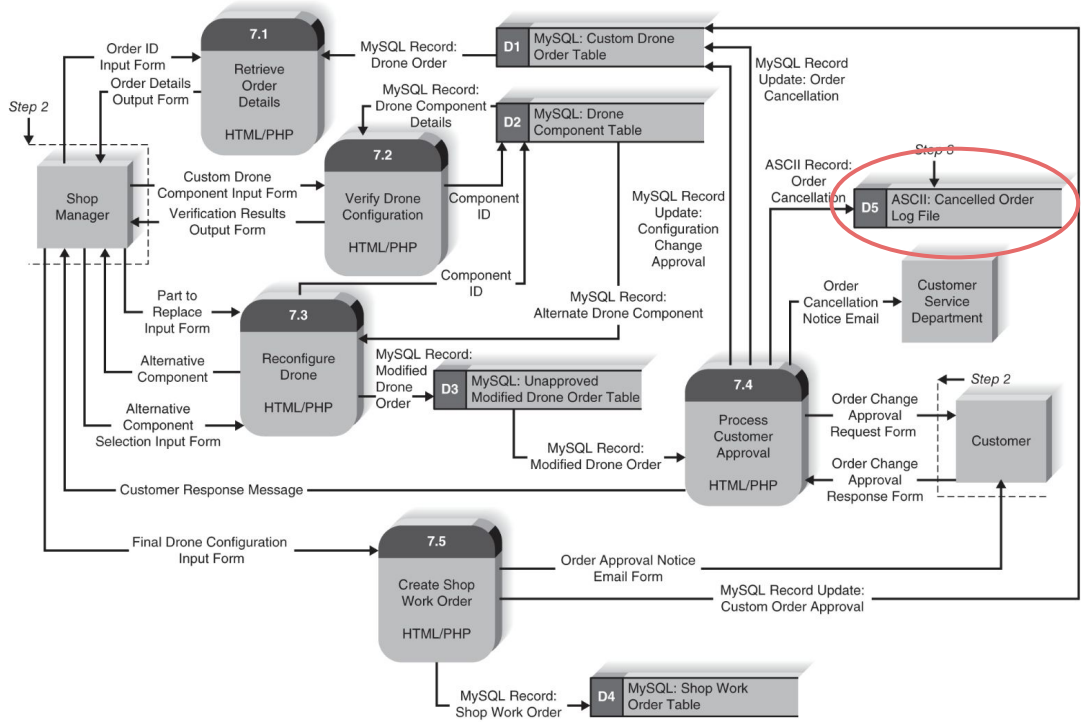


FIGURE 9-2 Physical DFD for shop manager approval of custom drone order level 1 DFD.

Creating Physical Data Models

1. Add implementation references
2. Draw a human-machine boundary
   a. Notice that not all external entities have a boundary around them. Why?
3. Add system related date stores, data flows, and processes.
4. Update the data elements in the data flows.
   a. E.g. last_updated or updated_by

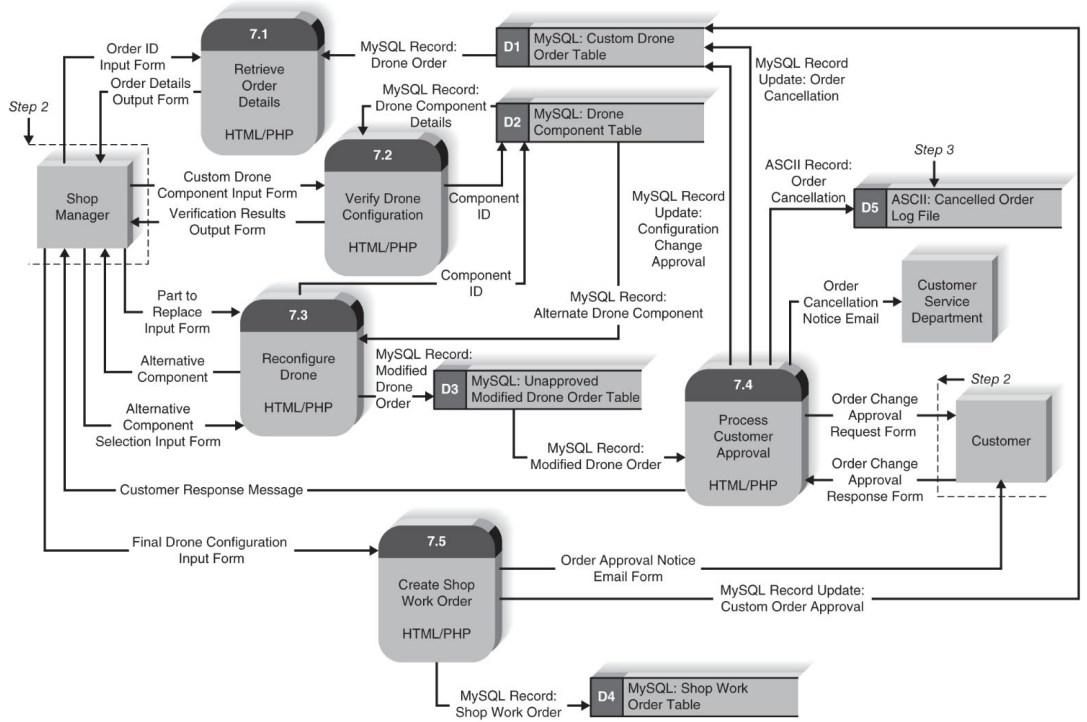


FIGURE 9-2 Physical DFD for shop manager approval of custom drone order level 1 DFD.

Creating Physical Data Models

1. Add implementation references
2. Draw a human-machine boundary
   a. Notice that not all external entities have a boundary around them. Why?
3. Add system related date stores, data flows, and processes.
4. Update the data elements in the data flows.
   a. E.g. last_updated or updated_by
5. Update the metadata in the CASE repository.
   a. E.g. when batch processes will be run and how often, names of actual tables or files, and the sizes and projected growth rates of data stores,

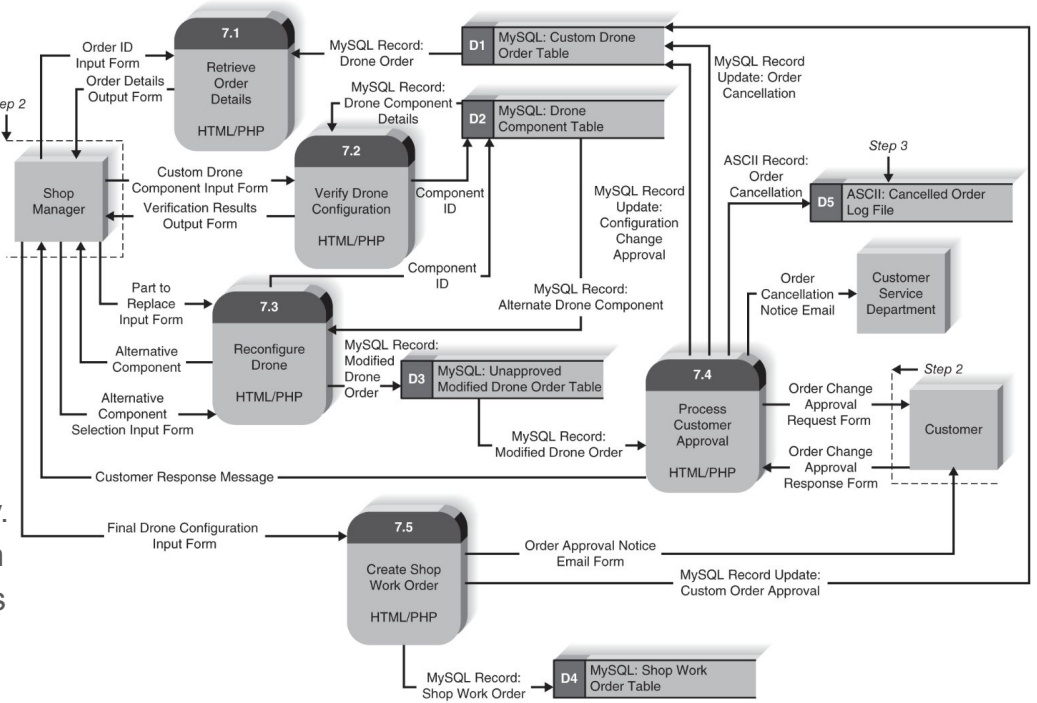


FIGURE 9-2 Physical DFD for shop manager approval of custom drone order level 1 DFD.

Step		Explanation
1.	Add implementation references.	Using the existing logical DFD, add the way in which the data stores, data flows, and processes will be implemented to each component.
2.	Draw a human–machine boundary.	Draw a line to separate the automated parts of the system from the manual parts.
3.	Add system-related data stores, data flows, and processes.	Add system-related data stores, data flows, and processes to the model (components that have little to do with the business process).
4.	Update the data elements in the data flows.	Update the data flows to include system-related data elements.
5.	Update the metadata in the CASE repository.	Update the metadata in the CASE repository to include physical characteristics.

FIGURE 9-1 Steps to create the physical data flow diagram.

❑ Your Turn: Logical vs. Physical DFDs

❑ Reading: Applying the Concepts at DronTeq

Designing Programs

Program Design

- There are *techniques* (Program Design Techniques) for designing *“Good Programs”*
  - What makes a program good?
- Program Design is still important because:
  - We need to understand existing codes
  - We need to re-organize/refactor code frequently to keep it good
  - We need to write original programs or piece together existing software systems

A Good Program

- Is easy to maintain
- Is modular
- Is flexible

Top-down modular approach (to program design)

1. Create a **high-level diagram** that shows:
   a. the various components of a program
   b. how the components are/should be organized
   c. How the components interrelate

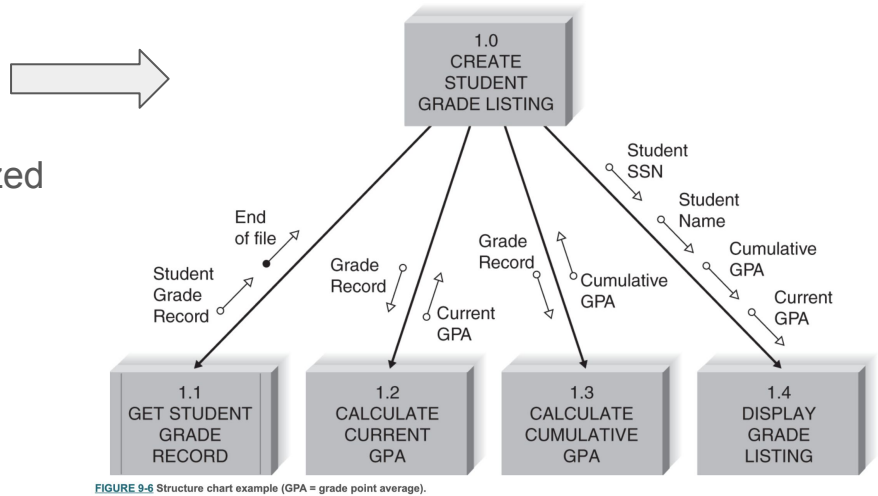


FIGURE 9-6 Structure chart example (GPA = grade point average).

Top-down modular approach (to program design)

1. Create a **high-level diagram** that shows:
   a. the various components of a program
   b. how the component are/should be organized
   c. How the components interrelate
2. Once the overall program is defined at a high level, we might go ahead and describe the **logic of the smaller modules**.

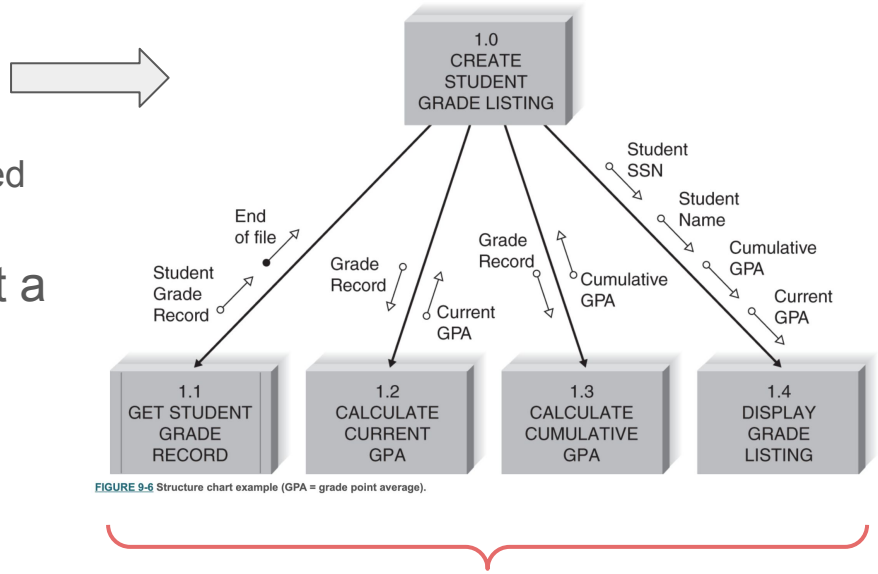


FIGURE 9-6 Structure chart example (GPA = grade point average).

Program Specs/ Functional Specs

Structure Chart

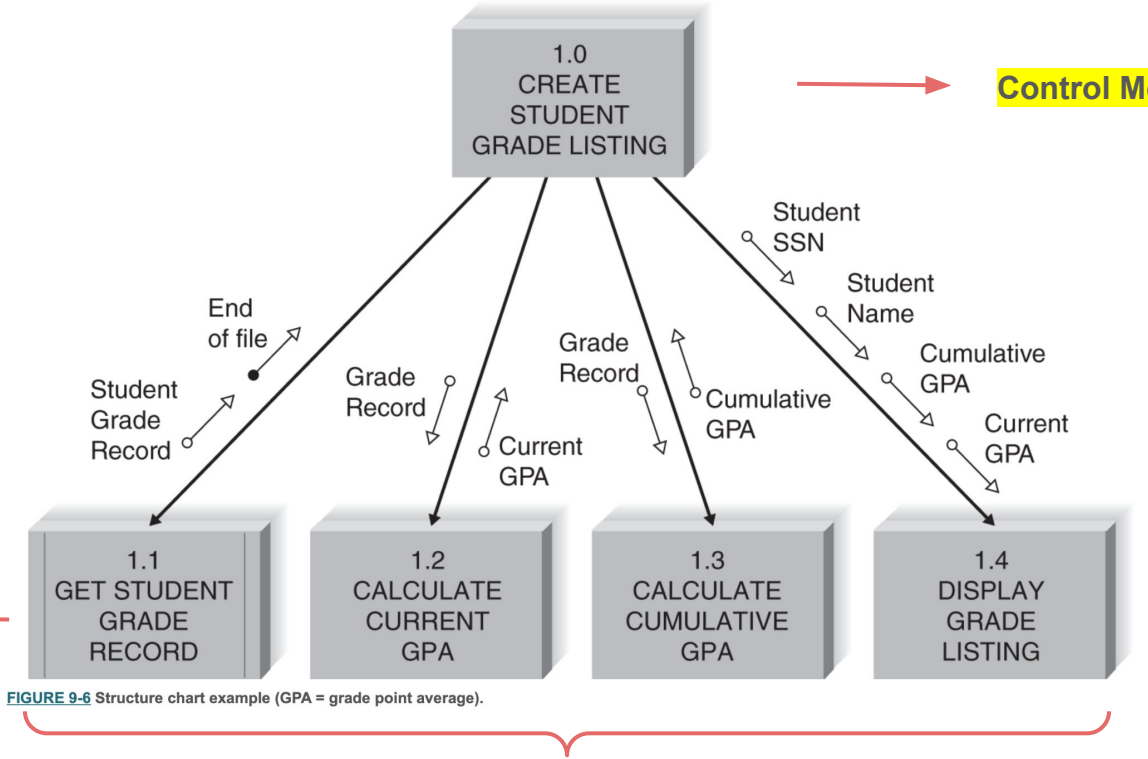


FIGURE 9-6 Structure chart example (GPA = grade point average).

Structure Chart

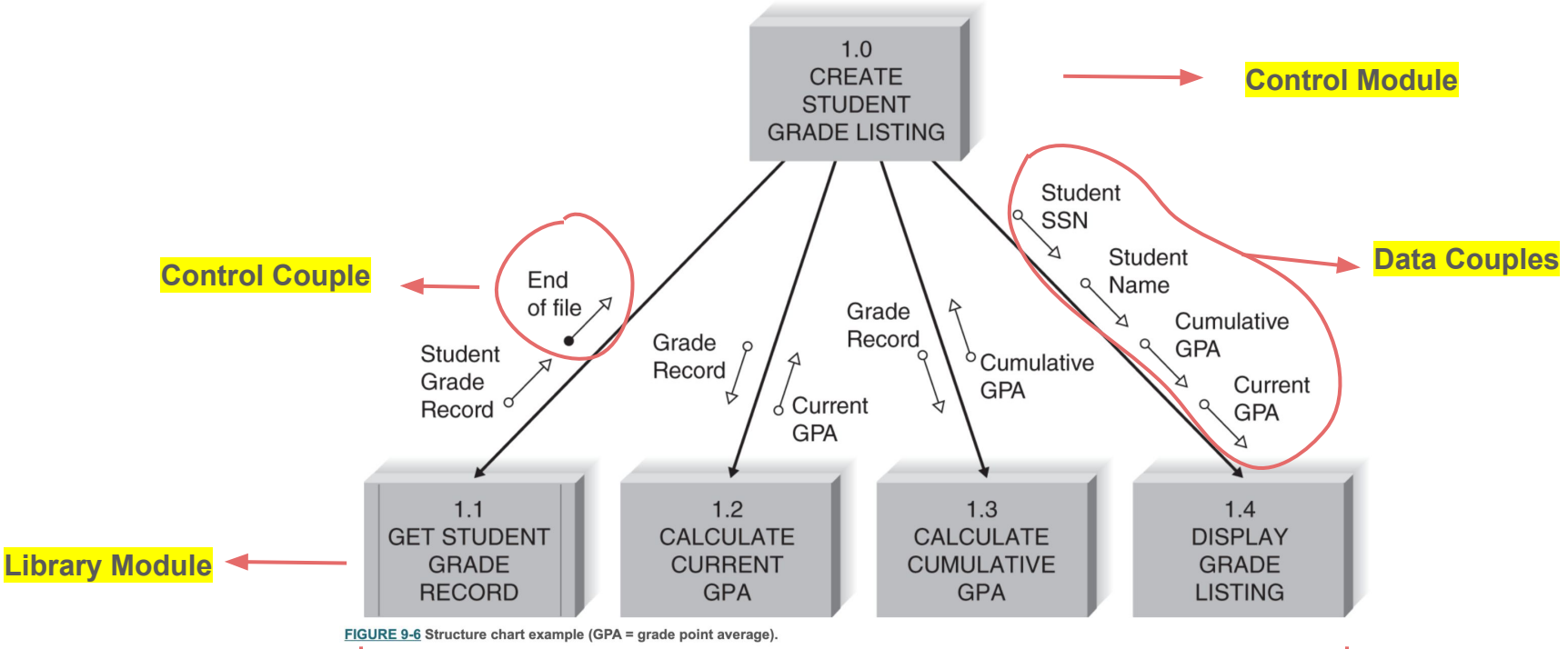


FIGURE 9-6 Structure chart example (GPA = grade point average).

Structure Chart

Structure charts not only imply **sequence (**Control module invokes submodules**),** they also show **selection** and **iteration** in programs.

Selection: subordinate modules are invoked by the control module based on some condition.

Iteration: module(s) is repeated.

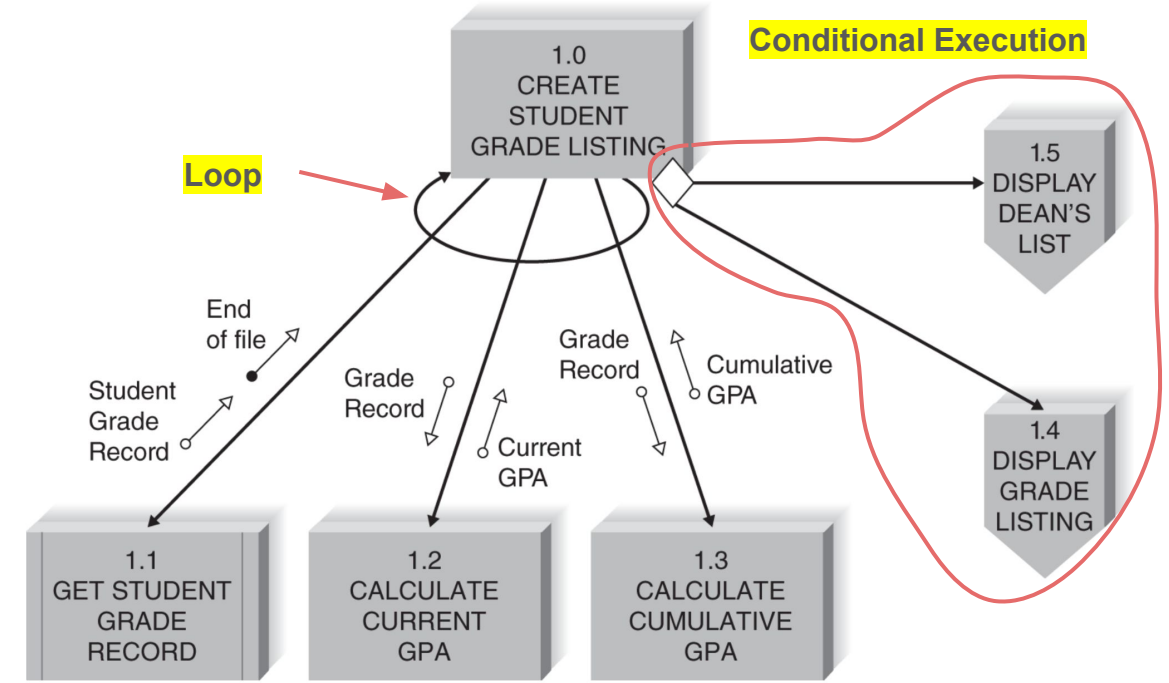


FIGURE 9-8 Revised structure chart example.

Building the Structure Chart

From DFD to Structure Charts

- DFDs are used as a **starting point** for structure charts
  - Each level of DFD → maps to a level of the structure chart
  - Each process of a DFD level → maps to a module on the structure chart

- **The Challenge: How should we organize** these modules to show sequence (who calls whom), selection (is the execution conditional?) and iteration (is there repetition)?

❑ Reading: Applying the concepts at DronTeq

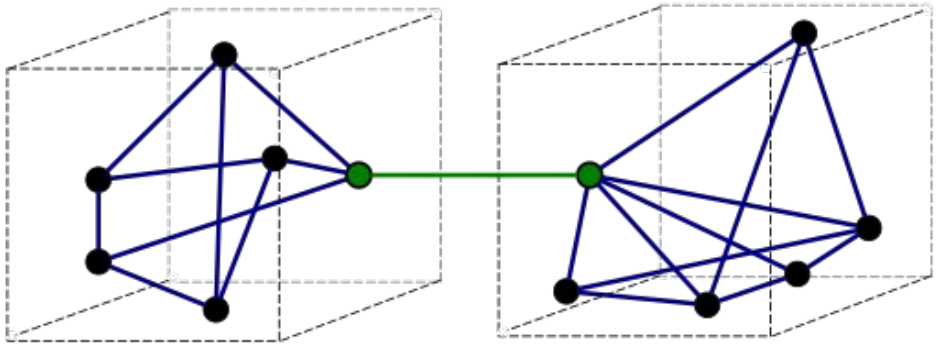
Design Guidelines

Build Modules with High Cohesion

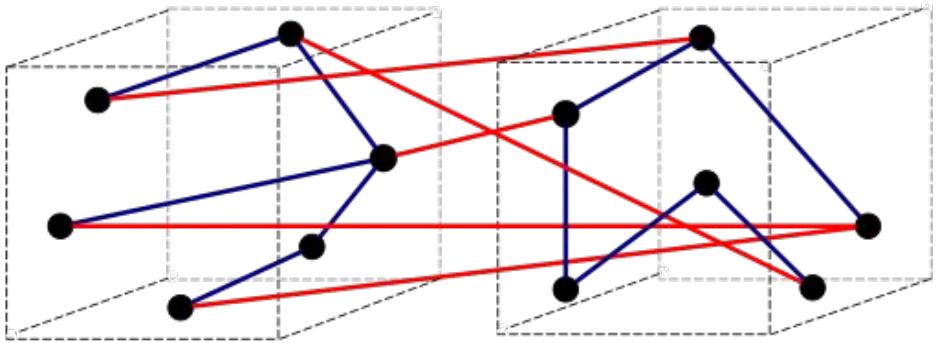
- **Cohesion:** How well the **lines of code in each module** relate to each other?
- **Ideally, a module should do only one thing and do it very well → high cohesion.**
- Highly cohesive modules are **easy to understand and build.**
- The more tasks that a module has to perform, the more complex the logic in the code must be.

Build Loosely Coupled Modules

- **Coupling:** How closely **different modules** are interrelated?
- Ideally modules should be **loosely coupled**.
- Loosely coupled modules **prevent changes** in one module **from rippling** throughout the program.



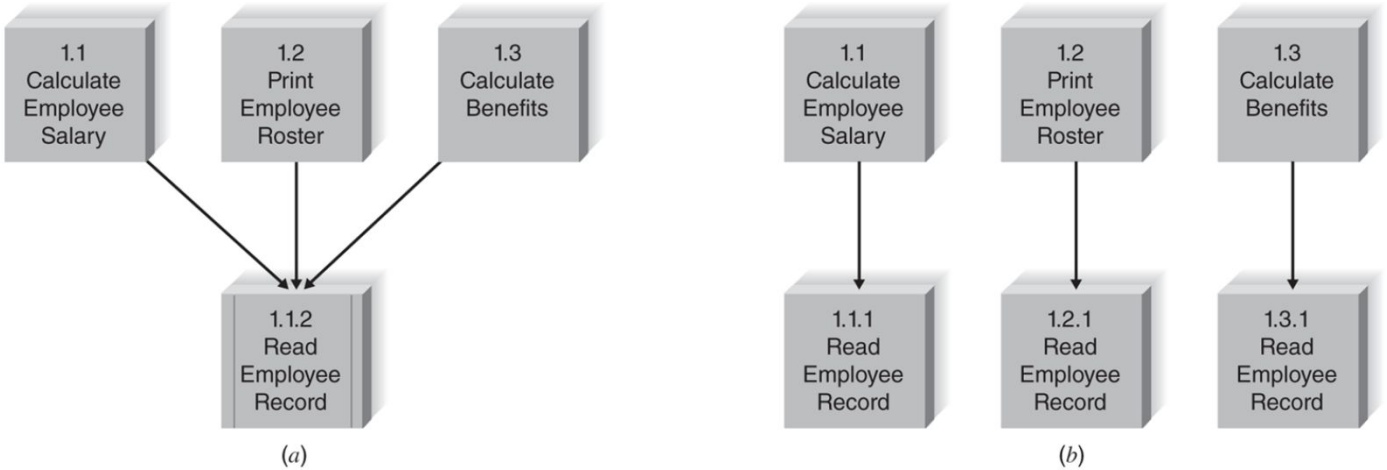
a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

Create High Fan-In

- **Fan-In:** The number of control modules that invoke a submodule.
- Ideally we want **high fan-in** modules → means the **module is reused** in many places.



Avoid High Fan-Out

- **Fan-out:** number of submodules that a control module would need to control.
- A control module would become much less effective when given large number of modules to control.--> **avoid high fan out**

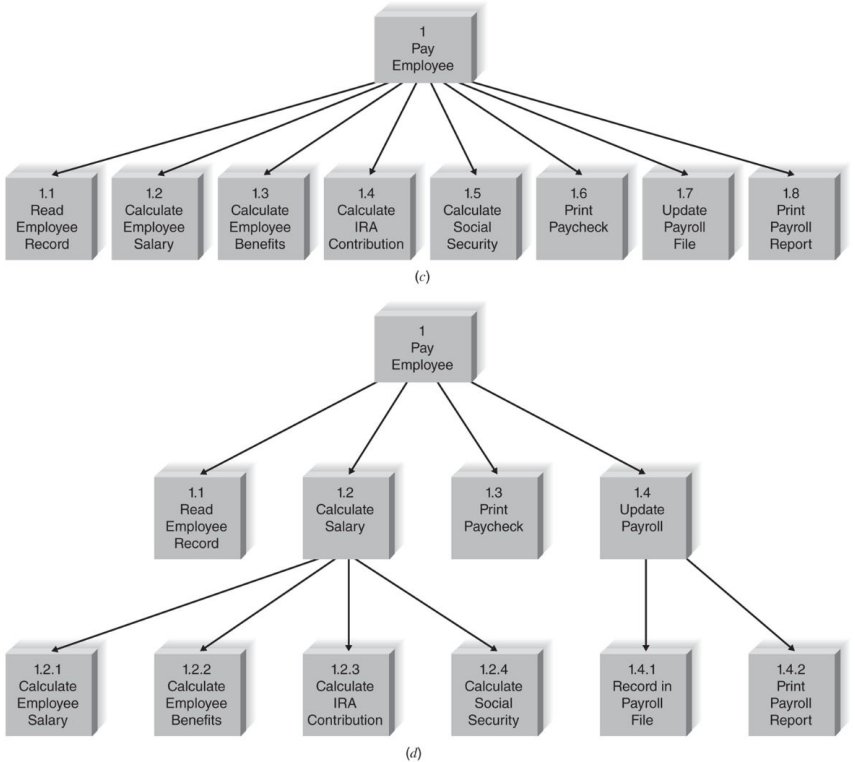


FIGURE 9-17 Examples of fan-in and fan-out: *(a)* high fan-in; *(b)* low fan-in; *(c)* high fan-out; *(d)* low fan-out (IRA = individual retirement account).

A Good Program

- Is easy to maintain
- Is modular
- Is flexible

Design Guidelines

1. Build modules with **high cohesion**
2. Build **loosely coupled** modules
3. Create high fan-in
4. Avoid high fan-out

- Your Turn: Different types of cohesion and coupling

Program Specification

How do we specify the logic that the module should implement?

- No formal syntax. Some companies use **specification forms** and **pseudocode**.
- Purpose: To describe the logic in enough details so the programmer can take over and write the code.

```
Calculate_discount_amount (total_price, discount_amount)
If total_price < 50 THEN
   discount_amount = 0
ELSE
If total_price < 500 THEN
   discount_amount = total_price *.10
ELSE
   discount_amount = total_price *.20
END IF
END
```

FIGURE 9-20 Pseudocode.

FIGURE 9-19 Program specification form.